

# Iteration Statements

How Java repeats work - and how we control when it stops

---

**for / while / do-while**

# Loops solve a maintenance problem before a speed problem

Repeated code works at first. It becomes fragile when the number of repetitions or the repeated action changes.

## WITHOUT A LOOP

```
System.out.println("Send reminder 1");  
System.out.println("Send reminder 2");  
System.out.println("Send reminder 3");  
System.out.println("Send reminder 4");  
System.out.println("Send reminder 5");
```

**The problem is not that Java cannot run this.**

The problem is that the pattern is copied five times.

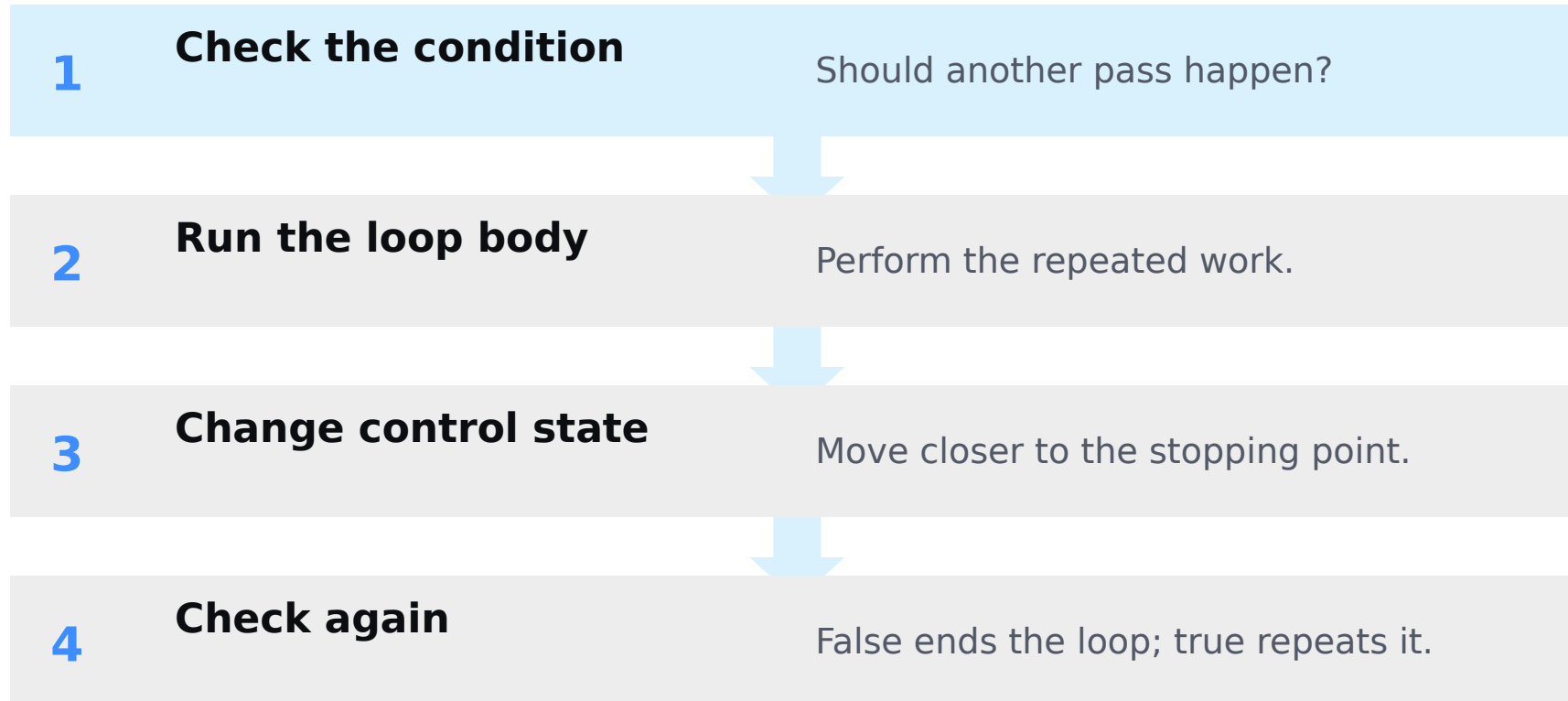
If the message changes, five lines must change. If the requirement becomes 500 reminders, the program needs 495 more lines.

A loop describes the changing part once and lets Java repeat the block.

**A loop replaces duplicated instructions with one controlled pattern.**

# A loop is a repeated decision - not just repeated code

Java checks a condition, runs a block when the condition is true, changes some control state, and checks again.



# Every safe loop answers three control questions

If any answer is missing, the loop may start incorrectly, include the wrong values, or never stop.

**01**

## Where does it start?

Choose the first control value before repetition begins.

**Start with reminder 1**

**02**

## When does it continue?

A boolean condition decides whether the next pass belongs.

**Continue through reminder 5**

**03**

## What changes each pass?

Progress must move the program toward a false condition.

**Move to the next reminder**

**A loop is correct only when its start, condition, and progress match the requirement.**

# Java has three basic loop structures because repetition begins differently

The best loop is the one whose structure matches what the requirement already tells you.

| Structure | Use it when...                                                 | First condition check | What to notice                        |
|-----------|----------------------------------------------------------------|-----------------------|---------------------------------------|
| for       | The control plan belongs together:<br>start, condition, update | Before the body       | Often used for a known range or count |
| while     | Repetition depends on a state that may already be false        | Before the body       | The body can run zero times           |
| do-while  | The work must happen once before Java can decide to repeat     | After the body        | The body always runs at least once    |

Enhanced for is introduced later when the requirement becomes: visit every value in an array or collection.

# A for loop keeps its control plan in one place

Initialization runs once. The condition is checked before every pass. The update runs after every completed pass.

## RUNNABLE JAVA PROGRAM

```
public class AttemptCounter {  
    public static void main(String[] args) {  
        for (int attempt = 1; attempt <= 3; attempt++) {  
            System.out.println("Attempt " + attempt);  
        }  
    }  
}
```

### START

**int attempt = 1**

Runs once

### CONTINUE

**attempt <= 3**

Checked before each pass

### PROGRESS

**attempt++**

Runs after each pass

# Read a for loop in execution order - not left to right forever

Only initialization happens once. The condition, body, and update form the repeating cycle.

| Step | attempt | Action                       | Result            |
|------|---------|------------------------------|-------------------|
| 1    | 1       | Initialize                   | Runs once         |
| 2    | 1       | Test $1 \leq 3$              | true - enter body |
| 3    | 1       | Print                        | Attempt 1         |
| 4    | 2       | Update, then test            | true - enter body |
| 5    | 2       | Print                        | Attempt 2         |
| 6    | 3       | Update, test, print          | Attempt 3         |
| 7    | 4       | Update, then test $4 \leq 3$ | false - stop      |

# The condition decides whether the end value counts

The loop body runs only for values that make the condition true. The first false result stops repetition immediately.

Control condition

**attempt <= 3**

The operator <= includes 3 because 3 <= 3 is true.

| attempt | attempt <= 3 | Body runs? |
|---------|--------------|------------|
| 1       | true         | yes        |
| 2       | true         | yes        |
| 3       | true         | yes        |
| 4       | false        | stop       |

**Changing <= to < would exclude 3. That is a requirement change, not merely a syntax preference.**



# The update controls the distance and direction of each step

A loop does not have to move by one. Its update should match the pattern described by the requirement.

ONE LOOP BODY

```
for (int seat = 2; seat <= 10; seat += 2) {  
    System.out.println("Check seat " + seat);  
}
```



2 4 6 8 10

**seat += 2 moves directly to the next even seat. Using seat++ would violate the pattern by visiting every seat.**

# Audit every third order

Translate the requirement into initialization, condition, update, and persistent state.

## REQUIREMENTS

- First eligible order ID: 2402
- Last eligible order ID: 2420, inclusive
- Audit every third ID
- Print each selected ID
- Print the final number of audits

## COMPLETE THE CONTROL PLAN

```
public class OrderAuditPractice {  
    public static void main(String[] args) {  
        int auditCount = 0;  
  
        for (int orderId = ____; ____; ____) {  
            System.out.println("Audit " + orderId);  
            ____;  
        }  
  
        System.out.println("Total audits: " + auditCount);  
    }  
}
```

**Explain why each blank matches the requirement before running the program.**

# The requirement supplies every part of the control plan

The solution is not a memorized loop template. Each expression comes directly from one sentence in the requirements.

## RUNNABLE SOLUTION

```
public class OrderAuditSolution {  
    public static void main(String[] args) {  
        int auditCount = 0;  
  
        for (int orderId = 2402; orderId <= 2420; orderId += 3) {  
            System.out.println("Audit " + orderId);  
            auditCount++;  
        }  
  
        System.out.println("Total audits: " + auditCount);  
    }  
}
```

**2402**

first eligible ID

**<= 2420**

inclusive final ID

**+= 3**

every third ID

**auditCount++**

one audit completed

**Expected total: 7 audits**

# A while loop repeats because a state is still true

Use while when the condition is the natural center of the requirement and the body may not need to run at all.

## Requirement

### Keep cleaning temporary files while available space is below 50 GB.

The loop does not need a separate pass number. Its controlling state is the amount of available space. Each pass must increase that state or the condition will never become false.

#### RUNNABLE JAVA PROGRAM

```
public class DiskSpaceCleanup {  
    public static void main(String[] args) {  
        int availableSpace = 25;  
  
        while (availableSpace < 50) {  
            System.out.println("Cleaning temporary files...");  
            availableSpace += 10;  
        }  
  
        System.out.println(  
            "Available: " + availableSpace + " GB"  
        );  
    }  
}
```

# A while loop checks before it performs the work

That first check matters: if the state already satisfies the requirement, the loop body runs zero times.

| Available space | Condition < 50 | Action                | Space after action |
|-----------------|----------------|-----------------------|--------------------|
| 25 GB           | true           | clean temporary files | 35 GB              |
| 35 GB           | true           | clean temporary files | 45 GB              |
| 45 GB           | true           | clean temporary files | 55 GB              |
| 55 GB           | false          | skip body and stop    | 55 GB              |

**If availableSpace starts at 55, the first condition is false and no cleanup runs. That is correct behavior.**

# A do-while loop performs the work once before checking

Use it when Java cannot evaluate the repeat condition until the first attempt has already produced a value.

## RUNNABLE JAVA PROGRAM

```
import java.util.Scanner;

public class RatingValidation {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        int rating;

        do {
            System.out.print("Rating 1-5: ");
            rating = input.nextInt();
        } while (rating < 1 || rating > 5);

        System.out.println("Accepted: " + rating);
        input.close();
    }
}
```

## Why the order fits

1. Java must ask for a rating at least once.
2. The entered value creates the state that can be validated.
3. Invalid input repeats the prompt; valid input ends the loop.

**In a web application, the input source changes; the validation decision remains.**

# Choose the loop from how repetition is controlled

Start with the requirement. Then choose the structure that makes the start, condition, and progress easiest to explain.

| Requirement signal                          | Choose       | Reason                                     |
|---------------------------------------------|--------------|--------------------------------------------|
| Repeat across a planned range or count      | for          | The complete control plan belongs together |
| Repeat while a state remains true           | while        | The condition is checked before any work   |
| Perform once, then decide whether to repeat | do-while     | The condition depends on the first result  |
| Visit every value in an array or collection | enhanced for | The requirement needs values, not indexes  |

Later in the full lesson: enhanced for, nested loops, break, continue, and debugging loops that do not stop.

# A loop is correct only when you can explain why it...

**starts**

the first state matches the requirement

**continues**

the condition includes exactly the intended passes

**changes**

every pass makes deliberate progress

**stops**

the condition eventually becomes false

**Syntax becomes useful after the control plan makes sense.**



# Now apply the control plan to data and branching

The loop structures stay the same. What changes is the kind of work performed during each pass.

---

# Enhanced for visits each value without an index

Use it when the requirement says every value and never asks for a position, replacement, or custom step.

## What changes

The collection supplies each value automatically. Java handles the starting position, boundary check, and one-step movement.

Read it as:

**For each service in services,  
run this block.**

### RUNNABLE JAVA PROGRAM

```
public class ServiceNames {  
    public static void main(String[] args) {  
        String[] services = {"Auth", "Billing", "Search"};  
  
        for (String service : services) {  
            System.out.println(service);  
        }  
    }  
}
```

# Enhanced for works when position does not matter

Giving up the index removes unnecessary machinery, but it also removes index-based control.

| Requirement                     | Best fit     | Why                                |
|---------------------------------|--------------|------------------------------------|
| Print every stored service name | enhanced for | Only each value is needed          |
| Replace the value at position 2 | indexed for  | The position is part of the result |
| Visit every second position     | indexed for  | The update must move by two        |
| Compare neighbors               | indexed for  | The program needs $i$ and $i + 1$  |

Use the simpler form only when it still exposes everything the requirement needs.

# Nested loops repeat one loop inside another

The outer loop selects a group. The inner loop processes every repeated item inside that group.

## OUTER LOOP

### Choose one rack

The rack number changes only after every slot in the current rack is processed.

## INNER LOOP

### Visit every slot

The slot counter starts over for each new rack because each rack owns a complete slot cycle.

**3 racks x 2  
slots = 6 body  
executions**

**Nested loops multiply combinations; they do not run as two unrelated loops.**

# The inner loop restarts for every outer pass

rack stays constant during the inner cycle; slot changes from 1 through 2 for that rack.

## RUNNABLE JAVA PROGRAM

```
public class RackSlotInspection {  
    public static void main(String[] args) {  
        for (int rack = 1; rack <= 3; rack++) {  
            for (int slot = 1; slot <= 2; slot++) {  
                System.out.println(  
                    "Rack " + rack + ", slot " + slot  
                );  
            }  
        }  
    }  
}
```

## Output order

```
R1 S1  
R1 S2  
R2 S1  
R2 S2  
R3 S1  
R3 S2
```

# Trace nested loops by recording both control values

Each row represents one execution of the innermost body.

| Body execution | rack | slot | Printed result |
|----------------|------|------|----------------|
| 1              | 1    | 1    | Rack 1, slot 1 |
| 2              | 1    | 2    | Rack 1, slot 2 |
| 3              | 2    | 1    | Rack 2, slot 1 |
| 4              | 2    | 2    | Rack 2, slot 2 |
| 5              | 3    | 1    | Rack 3, slot 1 |
| 6              | 3    | 2    | Rack 3, slot 2 |

# The inner loop completes before it starts again

number prints 1, 2, 3 during each round.

## RUNNABLE JAVA PROGRAM

```
public class RepeatNumbers {  
    public static void main(String[] args) {  
        for (int round = 1; round <= 2; round++) {  
            for (int number = 1; number <= 3; number++) {  
                System.out.println(number);  
            }  
        }  
    }  
}
```

## Output

1  
2  
3  
1  
2  
3

**The second 1 appears when the inner loop starts over for round 2.**

# Now predict a separate nested loop

The outer loop controls the week; the inner loop controls the day.

## RUNNABLE JAVA PROGRAM

```
public class WeekAndDay {  
    public static void main(String[] args) {  
        for (int week = 1; week <= 2; week++) {  
            for (int day = 1; day <= 3; day++) {  
                System.out.println(  
                    "Week " + week + ", Day " + day  
                );  
            }  
        }  
    }  
}
```

## Before running it

1. What prints first?
2. After Week 1 ends, what prints next?
3. How many lines print?



# break ends the nearest loop when work is complete

Use it when continuing would perform unnecessary or incorrect work after a decisive result is found.

## RUNNABLE JAVA PROGRAM

```
public class FindOrder {  
    public static void main(String[] args) {  
        int targetId = 1006;  
        boolean found = false;  
  
        for (int orderId = 1001; orderId <= 1010; orderId++) {  
            if (orderId == targetId) {  
                found = true;  
                System.out.println("Found order " + orderId);  
                break;  
            }  
        }  
  
        System.out.println("Search complete: " + found);  
    }  
}
```

## What break changes

Without break, the loop would keep checking IDs 1007 through 1010 even though the target was already found.

Control moves to the first statement after the loop.

**break does not end the method; it ends the nearest loop.**

# continue skips one pass and keeps the loop running

Use it when the current item should be ignored but later items still need processing.

## RUNNABLE JAVA PROGRAM

```
public class DeploymentSchedule {  
    public static void main(String[] args) {  
        for (int deployment = 1; deployment <= 6; deployment++) {  
            if (deployment == 4) {  
                System.out.println("Skip 4: maintenance window");  
                continue;  
            }  
  
            System.out.println("Run deployment " + deployment);  
        }  
    }  
}
```

## Pass 4

Print the skip message.

Bypass the normal deployment line.

Continue with pass 5.

# break and continue transfer control differently

Both interrupt the normal body, but only one ends the loop.

| Question            | Use break                                 | Use continue                               |
|---------------------|-------------------------------------------|--------------------------------------------|
| What happens now?   | The loop ends                             | Only this pass ends early                  |
| What happens next?  | Run the statement after the loop          | Begin the next loop pass                   |
| Typical requirement | Stop when the answer is found             | Ignore this item and keep processing       |
| Main risk           | Stopping before required work is complete | Skipping state updates the next pass needs |

Use either statement only when its control transfer makes the requirement clearer.

# A loop never ends if its condition cannot change

The code compiles, but pendingJobs never moves toward zero, so the condition never becomes false.

## BROKEN PROGRAM

```
public class MissingProgress {  
    public static void main(String[] args) {  
        int pendingJobs = 3;  
  
        while (pendingJobs > 0) {  
            System.out.println("Processing a  
job");  
        }  
    }  
}
```

## Start

pendingJobs = 3

## After every pass

still 3

**Repair the progress step: pendingJobs--;**

# A stray semicolon can become the entire loop body

Java interprets the statement ending at the semicolon as an empty repeated body.

COMPILES, BUT DOES NOT MEAN WHAT IT LOOKS LIKE

```
public class EmptyLoopBody {  
    public static void main(String[] args) {  
        int count;  
  
        for (count = 1; count <= 3; count++); {  
            System.out.println("Count: " + count);  
        }  
    }  
}
```

## What Java does

1. Repeat an empty statement three times.
2. Exit when count becomes 4.
3. Run the following block once.

Output: Count: 4

**Delete the semicolon after the for clause.**

# Debug a loop by checking the control plan in order

Do not stare at the entire program. Verify one control decision at a time.

- |   |                       |                                                         |
|---|-----------------------|---------------------------------------------------------|
| 1 | <b>Start</b>          | Is the first control value correct?                     |
| 2 | <b>Condition</b>      | Does true include exactly the intended passes?          |
| 3 | <b>Progress</b>       | Does every path move toward stopping?                   |
| 4 | <b>State lifetime</b> | Which answers must survive, and when should they reset? |
| 5 | <b>Early control</b>  | Can break or continue skip required updates?            |

# Complete a countdown from 5 to 1

Fill the condition and update so the loop stops after printing 1.

## REQUIREMENTS

- Begin at 5.
- Print the current number.
- Subtract 1 after each pass.
- Stop after printing 1.

## COMPLETE THE CONTROL PLAN

```
public class CountdownPractice {  
    public static void main(String[] args) {  
        int number = 5;  
  
        while (number ____ 1) {  
            System.out.println(number);  
            number = number ____ 1;  
        }  
    }  
}
```

**Explain how each blank matches the requirements.**

# The condition includes 1; the update subtracts 1

When number becomes 0, `number >= 1` is false.

## RUNNABLE SOLUTION

```
public class CountdownSolution {  
    public static void main(String[] args) {  
        int number = 5;  
  
        while (number >= 1) {  
            System.out.println(number);  
            number = number - 1;  
        }  
    }  
}
```

**Start**

`number = 5`

**Continue**

`number >= 1`

**Progress**

`number = number - 1`

**Stop**

`number becomes 0`

**The loop prints 5, 4, 3, 2, 1 - then stops.**



# Next, the loop gets a container of real values

**Arrays answer: where are the values stored?**

**Loops answer: how does Java visit and process them?**

The control questions do not disappear: start, continue, progress, state lifetime, and stop still determine whether the program is correct.

**The container changes. The reasoning stays.**